

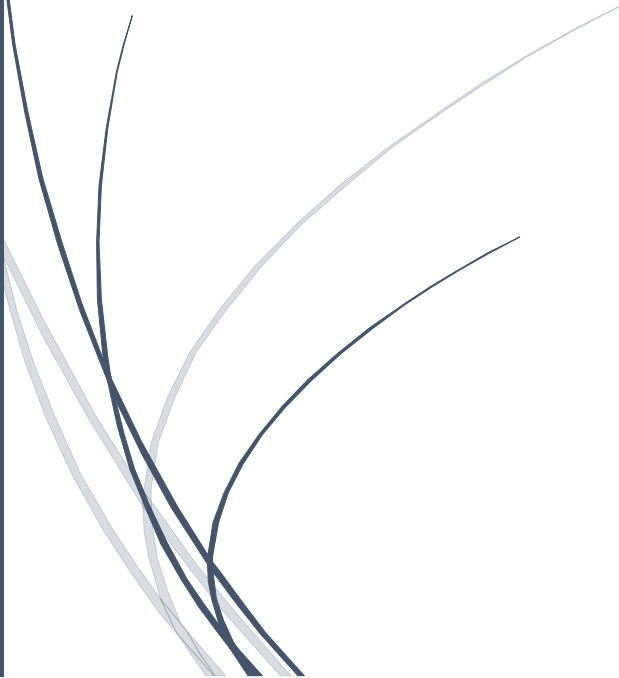


10-7-2026

UPA

Mongo DB

Basico Intermedio



Contenido

Instalación	2
Information	3
JavaScript	5
Backup and Restore	5
DDL.....	6
1. Create.....	7
2. Drop	9
3. Alter.....	9
CRUD	10
1. Insert	10
2. Delete.....	11
3. Update	12
4. Select.....	13
Aggregate	15
Inner join	17
Arreglos	18
Funciones	19
Cursores	20
Otros	20

→

Instalación

No.	S Q L	MongoDB
		Servidor: mongodb-windows-x86_64-8.3.4-signed.msi https://www.mongodb.com/try/download/community IDE: mongodb-compass-1.49.8-win32-x64.exe https://www.mongodb.com/try/download/compass MongoDB Shell: mongosh-2.9.2-x64.msi https://www.mongodb.com/try/download/shell
	Versiones anteriores	c:\>mkdir data c:\data>mkdir db
		SQL: DB, Tablas, Registros NoSQL: DB, Colecciones, Documentos (Objetos Json)
	Path	C:\Program Files\MongoDB\Server\5.0\bin
	Servidor Version Cliente Version	c:\>mongod -version c:\>mongos -version
	Conexion	Versiones Anteriores C:\> mongo mongodb://localhost:27017/Mexico C:\> mongo mongodb://localhost:27017/mydb -u user -p password Versiones Nuevas C:\> mongosh mongodb://localhost:27017 mongosh "mongodb://usuario:contraseña@localhost:27017/?authSource=admin" Conexion a Atlas(Cloud) mongosh "mongodb+srv://<usuario>@<cluster>.mongodb.net/"
	show databases	show dbs default (admin, config, local)

Information

MongoDB Compass: Manage your connection settings: `mongodb://admin:admin@localhost:27017/`

Localhost:27017

>_ Open MongoDB shell

+ Create database

↻ Refresh

>_ mongosh: localhost:27017

>_MONGOSH

> show dbs

< admin 100.00 KiB

config 72.00 KiB

local 80.00 KiB

prueba 324.00 KiB

test>

> use prueba

< switched to db prueba

> show collections

< departamentos

empleados

pcs

usuarios

prueba>

Connections Edit View Collection Help

Compass

{ My Queries

Data Modeling

Connections (1)

Search connections

localhost:27017

admin

config

local

prueba

departamentos

empleados

pcs

usuarios

>_ mongosh: localhost:27017

prueba

localhost:27017

empleados

{ My Queries

Data Modeling

+ Open MongoDB shell

localhost:27017 > prueba > empleados

Documents 5 Aggregations Schema Indexes 3 Validation

Type a query: { field: 'value' } or Generate query

Explain

Reset

Find

Options

25 1 - 5 of 5

_id: ObjectId('6a05fbefb70e75d74822ac09')

id: 1

nombre: "Carlos H."

email: "carlos.h@gmail.com"

fecha_creacion: 2026-05-14T10:44:31.921+00:00

fecha_modificacion: 2026-05-14T10:44:31.953+00:00

id_dep: 1

_id: ObjectId('6a05fbefb70e75d74822ac0a')

id: 2

nombre: "Juan Pérez"

email: "juan@gmail.com"

fecha_creacion: 2026-05-14T10:44:31.925+00:00

id_dep: 2



Muestra la DB activa	db
show databases	show dbs default (admin, config, local)
Crear o usar la DB mydb	use mydb
Comandos Generales	help
Comandos de db (versiones anteriores)	db. <tab> <tab> db.ciudades. <tab> <tab> db.ciudades.insert <tab> <tab>
	db.help()
Mostrar las Colecciones	show collections db.getCollectionNames()
Crea si no existe Colecciones	db.ciudades.insertOne({idciudad: 33, nombre: "Foraneos"})
	db.ciudades.help()
Contar los que comienzan con "F"	db.ciudades.countDocuments({nombre: /^F/})
Explain	db.ciudades.find({municipios:5}).explain("executionStats")
	db.empleados.countDocuments({hijos: {\$gte: 1}})

→

JavaScript

No.	S Q L	MongoDB
		Math.pow(2,3)
		"Hello World!".replace("World", "Mongodb")
	Funciones	var fnDoble = function (n) {return n*n} fnDoble(5)
		var stuff = [2,3,4,5] for (element of stuff) {print(element*2)}
		var x = {nombre:"Pepito", edad: 20} x.nombre

Backup and Restore

No.	S Q L	MongoDB
	Crear Respaldo Crear una carpeta dump/mydb	C:\> mongodump --db mydb
	Restaurar respaldo Dropear todas las colecciones	C:\> mongorestore --db mydb dump/mydb
	Respaldar collection en dump/mydb	C:\> mongodump --collection Ciudades --db mydb
	Restaurar collection Pero antes borrar coleccion ciudades	C:\> mongorestore --collection Ciudades --db mydb dump/mydb/Ciudades.bson

→

Operadores del lenguaje de consultas vs. opciones de configuración/flags.

◆ Operadores (siempre llevan \$)

Se usan dentro de consultas, actualizaciones o agregaciones. Ejemplos:

- **Lógicos:** `$and`, `$or`, `$not`
- **Comparación:** `$gt`, `$lt`, `$eq`
- **Tipos:** `$type`
- **Expresiones regulares:** `$regex`
- **Actualización:** `$set`, `$unset`, `$inc`
- **Agregación:** `$match`, `$group`, `$project`

👉 Regla: si es un operador que modifica o evalúa datos dentro de una consulta/operación, lleva \$.

Opciones / Flags (no llevan \$)

Son parámetros de configuración que acompañan a métodos como `createIndex()`, `update()`, `insert()`. Ejemplos:

- `unique` → define índice único
- `sparse` → índice que solo incluye documentos con el campo
- `upsert` → crea documento si no existe al hacer `update`
- `multi` → aplica actualización a múltiples documentos
- `validator` → reglas de validación al crear colecciones
- `background`, `expireAfterSeconds` → opciones de índices

👉 Regla: si es una opción de configuración o un flag que ajusta el comportamiento del comando, no lleva \$.

✅ Truco para no confundirte

Dentro de `find()`, `update()`, `aggregate()` → piensa en operadores → llevan \$.

Dentro de `createIndex()`, `updateOne()`, `updateMany()`, `createCollection()` → piensa en opciones/flags → no llevan \$.

DDL

1. Create

No.	S Q L	MongoDB
	Create DB	Use Mexico
	Create Collection	db.createCollection("empleados",
	Check o Validar	<pre>{ validator: { \$and: [{ nombre: { \$type: "string" } }, { sexo: { \$in: ["H", "M"] } }, { fec_nac: { \$type: "date" } }, { email: { \$regex: /@/ } }, { hijos: { \$type: "number", \$gte:0}}] } }</pre>
		<pre>db.createCollection("empleadotes", { validator: { \$jsonSchema: { bsonType: "object", required: ["nombre", "sexo", "fec_nac", "email", "hijos"], properties: { nombre: { bsonType: "string", description: "Debe ser una cadena" }, sexo: { enum: ["H", "M"], description: "Debe ser H o M" }, fec_nac: { bsonType: "date", description: "Debe ser una fecha" }, email: { bsonType: "string", pattern: "@", description: "Debe contener un @" }, hijos: {</pre>

		<pre> bsonType: "int", minimum: 0, description: "Debe ser un entero mayor o igual a 0" } } } } })) </pre>
	Create Collection e Insert	<pre> db.empleados.insertOne({ nombre: "Batman", sexo: 'H', fec_nac: new Date("2021/03/17"), email: "batman@upa", hijos: 2, ciudad: ObjectId("6a56ba1657c4c3413c0fcff9"), deportes:["basquet","futbol"], estatus: true }) </pre>
		<pre> db.empleados.insertMany([{ apellido: "García", nombre: "Carlos" }, { apellido: "García", nombre: "Ana" }]) </pre>
		Proyectos reales → _id como ObjectId + índice único en id_emp.
	Crear indices	<pre> db.empleados.createIndex({ apellido: 1 }) db.empleados.createIndex({ idEmpleado: 1 }, { unique: true }) </pre> Por default es Denso
	Crear indices compuestos	<pre> db.empleados.createIndex({ apellido: 1, nombre: 1 }) db.empleados.createIndex({ apellido: 1, nombre: 1 }, { unique: true }) </pre> Por default es Denso
	Mostrar Indices	db.Empleados.getIndexes();
		<pre> db.empleados.find({apellido: "Garcia", nombre: "Carlos"}) db.empleados.find({ \$and: [{apellido: "Garcia"}, {nombre: "Carlos"}] }) db.empleados.find({ \$or: [{apellido: "Garcia"}, {nombre: "Carlos"}] }) db.empleados.find({ \$nor: [{apellido: "Garcia"}, {nombre: "Carlos"}] }) </pre> AND implicito <pre> db.empleados.find({ \$and:[{ edad: { \$gt: 25 } }, { email: { \$regex: /gmail/ } }] }) </pre> > >= < <= = i= \$gt, \$gte, \$lt, \$lte, \$eq, \$ne, etc.



2. Drop

No.	S Q L	MongoDB
	Drop DB	<code>db.getSiblingDB("webstore").dropDatabase();</code>
	Drop DB actual	<code>db.dropDatabase()</code>
	Drop Collection	<code>db.getSiblingDB("testdb").getCollection("cars").drop();</code> <code>db.products.drop()</code>
	Eliminar Columna	<code>db.Empleados.update({}, {\$unset: {Telefono:1}} , {multi: true});</code>
	Drop Indices	<code>db.Orders.dropIndex({OrderID:-1})</code>

3. Alter

No.	S Q L	MongoDB
	Renombrar Coleccion	<code>db.Academicos.renameCollection("Maestros");</code>
	Todas las columnas	<code>db.ciudades.updateMany({},{\$rename: {"abreviatura": "abr"}});</code>
	Renombrar Columna del primero encontrado	<code>db.ciudades.update({}, { \$rename: { "Abreviatura": "Abr" } }, {multi:true})</code>

→

CRUD

1. Insert

No.	S Q L	MongoDB
		<pre>[var] doc = {username : "Pablo Milanes", age: 60} db.users.insert(doc)</pre>
		<pre>db.products.insert({ "nombre": "laptop", "precio": 40.2, "active": false, "created_at": new Date("2021/03/17"), "somedata": [1, "a", []], "facturer": { "name": "dell", "location": { "city": "usa", "address": "known" } } })</pre>
	Proyectos reales / producción: usar ObjectId como referencia (ciudad: <code>ObjectId(...)</code>) y mantener <code>id_cd</code> solo como campo adicional en ciudades.	<pre>db.empleados.insertOne({ nombre: "Batman", sexo: 'H', fec_nac: new Date("2021/03/17"), email: "batman@upa", hijos: 2, ciudad: ObjectId("6a56ba1657c4c3413c0fcff9"), id_cd: 1, deportes: ["basquet", "futbol"], estatus: true })</pre>
	<pre>id_cd: 1 db.empleados.aggregate([{ \$lookup: { from: "ciudades", localField: "id_cd", foreignField: "id_cd", as: "info_ciudad" } }])</pre>	<pre>ciudad: ObjectId("6a56ba1657c4c3413c0fcff9"), db.empleados.aggregate([{ \$match: { nombre: "Batman" } }, // filtra solo el empleado "Batman" { \$lookup: { from: "ciudades", // colección relacionada localField: "ciudad", // campo en empleados (ObjectId) foreignField: "_id", // campo en ciudades as: "info_ciudad" // nombre del arreglo resultante } }])</pre>



2. Delete

No.	S Q L	MongoDB
	Truncate coleccion	db.users.remove({})
	Eliminar documento	db.users.remove({"username": "Luis"})
		db.ciudades.deleteOne({_id: ObjectId("6197d9e68ba4caf6f940cb04")});
		db.ciudades.deleteOne({IdCiudad: 33});

→

3. Update

No.	S Q L	MongoDB
	Reemplaza el documento o registro completo	<code>db.products.update({"name":"keyboard"}, {"price":99.99})</code>
	Si no existe el atributo, lo agrega	<code>var usuario = db.users.findOne({"username":"Luis"}) usuario.age = 25 db.users.save(usuario)</code>
	Un solo registro	<code>var usuario = db.users.findOne({"username":"Luis"}) print(usuario) usuario.cp = 30000 db.users.update({"username":"Luis"}, usuario)</code>
	Upsert, Si no existe inserta registro Modifica los campos deseados	<code>db.ciudades.update({ IdCiudad: 33}, { \$set: { "Capital": "Meridiano", abr : "FOR"}} } , { upsert: true, multi:true})</code>
	Update ciudades set mun=mun+1 Where idciudad = 31;	<code>db.ciudades.updateOne({ IdCiudad: 31},{ \$inc:{Municipios:1}}) db.ciudades.updateOne({_id:ObjectId("6a554ef2ea1980c30eeb806f")} , { \$inc:{Municipios:1}})</code>



4. Select

No.	S Q L	MongoDB
	Select * from ciudades Order by id desc	db.ciudades.find({}) .projection({Capital:1, _id:0}) .sort({_id:-1}) .limit(100)
		db.Ciudades.findOne()
		db.Ciudades.find({Abreviatura: /^C/})
		db.Ciudades.count({Nombre: /^C/}); db.Ciudades.find({Nombre: /^C/}).count();
	Select * from ciudades Where mun > 50 And nombre like "C%"	db.ciudades.find({ Municipios: { \$gt: 50 }, Nombre : /^C/ });
	Select * from ciudades Where Mun > 100 Or Nom = "Aguascalientes"	db.ciudades.find({ "\$or": [{ Municipios: { \$gt: 100 } }, { Nombre : "AGUASCALIENTES" }] });
	Select * from ciudades Where Nombre = Capital	db.ciudades.find({ "\$where": "this.Nombre == this.Capital" });
	select * from Ciudades where Mun > 50 or Nombre != Capital	db.ciudades.find({ "\$or": [{ Municipios: { \$gt: 50 } }, { "\$where": "this.Nombre != this.Capital" }] });
	select * from Ciudades where Municipios > 100 and Nombre != Capital and Abreviatura != "Mex" Order by Capital desc Limit 3	db.ciudades.find({ Municipios: { \$gt: 100 }, "\$where": "this.Nombre != this.Capital", Abreviatura: { \$ne: "Mex" } }).sort({ Capital: -1 }).skip(1).limit(4).pretty()
	Mostrar ciudades que tengan el campo de Gobernador	db.ciudades.find({"Gobernador": {\$exists: true}})
	Select IdCiudad, Abr from ciudades Where abr not in ("Ags", "Zac")	db.ciudades.find({Abreviatura: {\$nin:["Ags", "Zac"]}}, {_id:0, IdCiudad:1, Abreviatura:true});

	Select * from Orders Where orderdate between x and y	<pre> db.Orders.find({ OrderDate: { \$gte: new Date("1998-04-21"), \$lt: new Date("1998-04-22") } }) </pre>
	Select * from orders Where year(OrderDate) = 1997	<pre> db.Orders.find({ "\$expr": { "\$eq": [{ "\$year": "\$OrderDate" }, 1997] } }) </pre>

→

Aggregate

N o.	S Q L	MongoDB
	Select count(distinct EmployeeID) From Orders;	db.Orders.distinct("EmployeeID").length
	SELECT distinct IdCiudad FROM ciudades Order by IdCiudad desc	db.ciudades.aggregate([{\$group: {_id: {IdCiudad: "\$IdCiudad"}}}, {\$project: {IdCiudad: "\$_id.IdCiudad" , _id: 0}}, {\$sort: { IdCiudad :-1}}])
	Select IdCiudad, count(*) From ciudades Group by IdCiudad Order by count(*) desc	db.ciudades.aggregate([{\$group: {_id: "\$IdCiudad", "Conteo": { \$sum: 1 }}}, {\$sort: {Conteo:-1}}])
	Select year(OrderDate), count(*) From Orders Group by year(OrderDate)	db.Orders.aggregate([{ \$group: { _id: { year: { \$year: "\$OrderDate" }}, conteo: {\$sum: 1}}}, { \$sort: {"_id.year":1}}])
	Select year(OrderDate), month(OrderDate), count(*) From Orders Group by year(OrderDate), month(OrderDate) Order by 1,2	db.Orders.aggregate([{ \$group: { _id: { year: { \$year: "\$OrderDate" }, Mes: {\$month: "\$OrderDate"}}}, conteo: {\$sum: 1}}}, { \$sort: {"_id.year":1, "_id.Mes":1 }}])
	Select OrderId, sum(UnitPrice*Quantity) From OrderDetails Group by OrderID Order by OrderId	db.OrderDetails.aggregate({ \$group: { _id: "\$OrderID", Importe: { \$sum: { \$multiply: ["\$UnitPrice", "\$Quantity"] } } } }).sort({"_id": 1})
	select sum(Quantity*UnitPrice*(1-Discount)) Total FROM OrderDetails	db.OrderDetails.aggregate([\$group: { _id: null, Total: { \$sum: { \$multiply: [{ \$multiply: ["\$UnitPrice", "\$Quantity"] }], { \$subtract: [1, "\$Discount"] } } } }])

		<pre> } } }]])</pre>
<pre>select OrderID, sum(Quantity*UnitPrice*(1-Discount)) Total FROM OrderDetails Group by OrderID Order by OrderID</pre>		<pre>db.OrderDetails.aggregate({ \$group: { _id: "\$OrderID", Total: { \$sum: { \$multiply: [{ \$multiply: ["\$UnitPrice", "\$Quantity"] }, { \$subtract: [1, "\$Discount"] }] } } } }).sort("_id":1)</pre>
<pre>select idCiudad, count(*) from Municipios where idCiudad != 31 group by idCiudad having count(*) >100 order by count(*);</pre>		<pre>db.Municipios.aggregate([{\$match: {IdCiudad: {\$ne: 31}}}, {\$group: {_id: "\$IdCiudad", "Conteo": { \$sum: 1 }}}, {\$match: {"Conteo": {\$gt:100}}}, {\$sort: {"Conteo":1}}])</pre>

→

Inner join

No.	S Q L	MongoDB
	<pre>Select C.IdCiudad, Abr, Nombre_Mun from Ciudades C inner join Municipios M on C.IdCiudad = M.IdCiudad where C.IdCiudad = 1</pre>	<pre>db.Ciudades.aggregate([{\$match: {IdCiudad:1}}, {\$lookup: {from: "Municipios", localField:"IdCiudad", foreignField:"IdCiudad", as: "Mun"}}, {\$unwind: "\$Mun"}, {\$project: {_id:0, IdCiudad:1, Abr:1, Municipio: "\$Mun.Nombre_Mun"}}])</pre>
	<pre>Select IdCiudad, Abr, Count(*), sum(Localidades) From Ciudades C, Municipios M Where C.IdCiudad = M.IdCiudad And C.IdCiudad = 1 Group by IdCiudad, Abr</pre>	<pre>db.Ciudades.aggregate([{\$match: {IdCiudad:1}}, {\$lookup: {from: "Municipios", localField:"IdCiudad", foreignField:"IdCiudad", as: "Mun"}}, {\$unwind: "\$Mun"}, {\$group: { _id: { IdCiudad: "\$IdCiudad", Abr: "\$Abr" }, Tot_Mun: { \$sum: 1 }, Tot_Loc: { \$sum: "\$Mun.Localidades" } }}])</pre>



Arreglos

No.	S Q L	MongoDB
	Los vectores comienzan en la posicion 0.	<pre>db.usuarios.drop() var arreglo= [11,2,3] var usuario = {Nombre:"Carlos", valores: arreglo} db.usuarios.insert(usuario)</pre>
	Agregar sin duplicados	<pre>db.usuarios.update({}, {\$addToSet: {valores: 3}})</pre>
	Agregar con duplicados	<pre>db.usuarios.update({}, {\$push: {valores: 3}})</pre>
	Agregar varios	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [3,4]}}})</pre>
	Agregar a partir de la posicion	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [10,20], \$position: 1 }}})</pre>
	Agregar y ordenar desc	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [10,20], \$sort:-1}}})</pre>
	Ordenar	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [], \$sort:-1}}})</pre>
	Eliminar el Numero	<pre>db.usuarios.update({}, {\$pull: {valores:11}})</pre>
	Eliminar por condicion	<pre>db.usuarios.update({}, {\$pull: {valores:{ \$gt:10}}})</pre>
	Eliminar lista	<pre>db.usuarios.update({}, {\$pullAll: {valores:[11,3]}})</pre>
	Eliminar la posicion 0	<pre>var data = db.usuarios.findOne({"Nombre":"Carlos"}) data.valores.splice(0,1) db.usuarios.save(data)</pre>
	Regresa la posicion	<pre>usuario.valores.indexOf('11') ??</pre>
	Seleccionar elemento	<pre>db.usuarios.find({},{valores:{\$slice:-1}})</pre>
	Seleccionar rango	<pre>db.usuarios.find({},{valores:{\$slice:[0,2]}})</pre>



Funciones

No.	S Q L	MongoDB
		function factorialR(n) { if (n <= 1) return 1; return n*factorialR(n-1) }
	between	db.products.remove({precio: {\$gt: 1, \$lt:99 }})

→

Cursores

No.	S Q L	MongoDB
		db.Ciudades.find().forEach(ciudad => print("Ciudad: " + ciudad.IdCiudad, "\t", ciudad.Abreviatura, "\t", ciudad.Nombre))
		for (i = 0; i < 5; i++) { db.products.insert({ "precio": i }) }
	Update 1X1 Cursor es valido solo una vez	var cursor = db.products.find() cursor.forEach((it) => { it.precio = it.precio + 1; db.products.save(it); });
	Update 1X1	var cursor = db.products.find() cursor.forEach(function(it) { it.precio = it.precio + 1; db.products.save(it); })

Otros

No.	S Q L	MongoDB
	Buscar en Arreglos	db.Restaurant.find({"grades.grade": "Z", "grades.score":20}).count()
	\$elemMatch en la misma posicion	db.Restaurant.find({grades: {\$elemMatch: {grade:"Z", score: 20}}}).count()
	Muestra solo el grado	db.Restaurant.find({"grades.grade":"Z"}, {_id:0, "grades.\$":1})

→